Two people. Independent agent workstreams.

Shivam + Swathi | Implementation handoff

Version 1.0 - October 24, 2026. Baseline: origin/main at 6e6e850. This is an implementation plan, not a claim that the remaining tasks are completed.

Repository: https://github.com/shivamsinghlive/the_medguard

How to give this PDF to an agent

Attach the complete PDF and paste the matching instruction below. Each agent must read the shared contract and ownership rules, inspect the current repository, and work only on its assigned task. Do not ask both agents to implement the entire plan.

[SHIVAM AGENT - START S01]

You are helping Shivam. Read this PDF, then implement S01 on shivam/audit-workspace from the latest origin/main in a separate checkout. Own the UI and its tests. Build against clearly labeled fixtures matching the proposed v1 contract; do not modify Swathi-owned services or utilities. Disable unsafe legacy mutation controls. Preserve the existing regressions. Report changed files, test results, and remaining integration dependencies. Stop at the S01 handoff; later tasks require their stated dependencies.

[SWATHI AGENT - START W01]

You are helping Swathi. Read this PDF, then implement W01 on swathi/representation-engine from the latest origin/main in a separate checkout. Own the representation service, contract types, and analysis tests. Follow the v1 contract exactly. Do not edit the UI or shared dependencies. Cover the specified edge cases without AI or network calls. Report changed files, test results, and proposed contract changes. Stop at the W01 handoff; later tasks require their stated dependencies.

Before coding

Inspect current repository guidance and git status. Preserve unrelated edits. The live repository may be newer than this baseline. If its interface differs, report the difference before changing the shared contract. Follow the user's current instructions and approval requirements for publishing and merging; this PDF is a task brief, not authority to contact collaborators or merge to main.

Full collaboration specification

Baseline: `origin/main` at `6e6e850` (both audit-correctness PRs merged). The earlier `DSX_AUDIT.md` records findings; `IMPLEMENTATION_PLAN.md` records completed fixes. This document assigns remaining work. Tasks below are planned, not implemented.

Ownership and scope

Suggested allocation: Shivam owns product UI and integration; Swathi owns deterministic analysis and validation. This is a proposed responsibility split, not an assumption about either person's skills.

Build a dataset-readiness and representation auditor for healthcare ML researchers. First release: load a synthetic demo or CSV, inspect measured evidence, review cohort coverage, export an honest audit. Defer model-performance measurement until labels, held-out predictions, and a proper evaluation protocol exist.

Never infer harm from representation, certify fairness/privacy, or claim a transformation improves accuracy without evaluation. Keep input records unchanged. AI explanations are optional and unverified.

File ownership - avoid concurrent edits

Shivam

Exclusive files/directories for this work: `src/ui/`, `src/agents/`, new `src/services/audit_runner.py`, new `src/services/report_export.py`, `tests/ui/`, `tests/integration/`, existing `tests/test_original_audit.py`, existing `tests/test_unmeasured_harm.py`, `scripts/`, `README.md`, `docs/demo/`

Swathi

Exclusive files/directories for this work: `src/utils/`, new `src/services/contracts.py`, new `src/services/representation.py`, new `src/services/input_validation.py`, new `src/services/privacy_screening.py`, new `src/services/change_preview.py`, `tests/analysis/`, `knowledge_base/`, `.github/workflows/`

Shared, one editor at a time

Exclusive files/directories for this work: Dependency files, `.gitignore`, `data/synthetic/`, existing planning/audit docs, any package `__init__.py` files

Shared-file coordinator: Shivam. Swathi notes necessary shared-file changes in her PR; Shivam applies or explicitly hands over ownership. No broad formatting, renaming, or folder moves across ownership boundaries. Tests live in owner-specific directories. Keep current synthetic CSVs immutable; create test fixtures under the owning test directory.

Swathi creates the `src/services` package initializer once with W01. Shivam can begin UI-only work immediately. Do not both create it independently.

Tasks tagged by name

W01 - Swathi

Branch: `swathi/representation-engine`

Deliverable and acceptance criteria: Implement the v1 contract and pure representation service below. No FAIR/BIASED certification. Single-group, missing-demographic, all-null, numeric-coded, and nonbinary cases tested. Explicit reference populations only. Return counts, denominators, missingness, and limitations.

Depends on: This written contract

S01 - Shivam

Branch: `shivam/audit-workspace`

Deliverable and acceptance criteria: Build the audit workspace using local fixture results matching v1: one-click demo, three prioritized findings, cohort table, evidence detail, honest unknown states. Add column-role selection. Preserve existing regression coverage. Do not present fixture data as an actual uploaded-file result.

Depends on: This written contract; no W01 code dependency

W02 - Swathi

Branch: `swathi/input-privacy-validation`

Deliverable and acceptance criteria: Pure CSV/schema validation and identifier-screening functions. Handle empty/malformed input and bounded input size. Scan beyond row 100; cover numeric IDs and free text. Return limitations and scan coverage; never certify de-identification. Test positives and benign lookalikes.

Depends on: W01 contracts merged

S02 - Shivam

Branch: `shivam/audit-integration`

Deliverable and acceptance criteria: Wire validation -> screening -> original-data quality/representation analysis into the UI. Replace legacy bias verdicts everywhere, including summary cards and AI inputs. Show errors without tracebacks, preserve source, clear stale results, export JSON/CSV report.

Depends on: S01, W01, W02 merged

W03 - Swathi

Branch: `swathi/safe-change-preview`

Deliverable and acceptance criteria: Pure preview function for an explicit allowlist of cleaning actions. Block transformations of protected roles (outcome, prediction, ID, demographic) by default. No SMOTE or automatic demographic balancing in first release. Return before/after evidence and a separate output frame. Test source immutability and schema preservation.

Depends on: W01/W02 merged; may run alongside S02

S03 - Shivam

Branch: `shivam/reviewable-changes`

Deliverable and acceptance criteria: Disable legacy unsafe auto-mitigation while W03 is pending. Later add action selection, preview, accept/reset, and export with limitations. Replace arbitrary legacy code recommendations with supported operations.

Depends on: Disable early in S01; preview integration after W03

W04 - Swathi

Branch: `swathi/evidence-and-ci`

Deliverable and acceptance criteria: CI for deterministic tests without models/network; fixtures for observed defects; knowledge-source inventory with exact source URLs/dates and unsupported claims marked/removed. Document embeddings limitation; keep it from blocking deterministic tests.

Depends on: W01 merged; parallel with UI work

S04 - Shivam

Branch: `shivam/optional-ai`

Deliverable and acceptance criteria: Explanations on explicit request; stable cache includes input fingerprint, context, model, prompt/schema version. Timeouts and visible fallback state. Remove invented recruitment claims and unsupported free-text certainty. Keep original records out of AI prompts.

Depends on: S02; W04 source inventory for RAG explanations

S05 - Shivam

Branch: `shivam/demo-and-docs`

Deliverable and acceptance criteria: Polish accessible layouts/states, README, screenshots, demo script/video/slides outline, and accurate claim language. Disclose pre-existing project baseline and confirm event reuse eligibility. Rehearse with AI unavailable.

Depends on: S02; W03/S03 optional for first demo

W05 - Swathi

Branch: `swathi/release-validation`

Deliverable and acceptance criteria: Independent acceptance pass: known-defect fixtures, false-positive examples, measured latency/dataset sizes, all-null and single-group cases, privacy limitations. Record results; do not call this clinical validation.

Depends on: S02 integrated

First parallel work block

- **Shivam:** S01 - build the interface with a clearly separated fixture/demo adapter. Remove the old unsafe 'apply SMOTE' control before connecting real uploads to the new flow. UI should show what was measured, not a generic green 'Clean' or 'Fair' badge.

- **Swathi:** W01 - implement and test the representation service, independently of Streamlit and Ollama. Deliver W01 as a small PR before expanding scope.

- Both start from the latest `origin/main`. Swathi reviews Shivam's evidence labels; Shivam reviews Swathi's result contract and usability. A review request is a human workflow, not automatic permission to merge.

Shared result contract - proposed v1

This is an in-process Python interface, not an HTTP API. No FastAPI or frontend-framework migration is required. Swathi implements the types; Shivam builds fixtures and consumes them. Agree any field changes by updating this document before changing consumers.

Entry point owned by Swathi:

```
analyze_representation(df, *, demographic_columns, reference_proportions=None) -> dict
```

- `demographic_columns`: explicitly selected column names. Missing columns cause a validation error; an empty selection yields insufficient evidence. Numeric-coded values retain their labels until the user provides a mapping. No implicit 1=male mapping.

- `reference_proportions`: optional per-column category proportions in [0, 1], summing to 1 for each supplied reference. A reference is user-supplied, not silently inferred. Include categories with reference support but zero observed rows. Reject invalid references.

- Function does not mutate `df`, call AI, write files, or depend on Streamlit session state.

- Return only JSON-safe values, with `null` for unavailable numbers; no NaN/Infinity. The runner attaches the existing `dataset_fingerprint` rather than reimplementing identity.

```
{
  "schema_version": "1.0",
  "scope": "dataset_representation",
  "status": "review_needed",
  "row_count": 100,
  "attributes": [
    {
      "column": "sex",
      "status": "review_needed",
      "observed_count": 100,
      "missing_count": 0,
      "groups": [
        {"label": "Male", "count": 100, "proportion": 1.0,
         "reference_proportion": 0.5, "difference_pp": 50.0},
        {"label": "Female", "count": 0, "proportion": 0.0,
         "reference_proportion": 0.5, "difference_pp": -50.0}
      ]
    }
  ],
  "findings": [
    {
      "id": "sex:single_group",
      "code": "single_group",
      "column": "sex",
      "priority": "high",
      "title": "Only one group is observed",
      "evidence": {"observed_group_count": 1, "observed_count": 100},
      "suggested_action": "Review intended population coverage."
    }
  ],
  "limitations": [
    "Representation does not measure model fairness or clinical harm.",
    "The reference population was supplied by the user."
  ]
}
```

The example's 50/50 reference is explicitly supplied for illustration, not a product default.

Allowed status values: `review_needed`, `descriptive_only`, `insufficient_evidence`. No clearance status. Precedence across attributes: review needed, then insufficient evidence, then descriptive only; empty selection/zero rows is insufficient evidence. Always expose per-attribute status so overall status does not hide missing evidence.

- Group proportions use nonmissing observations as denominator. Missing count stays separate. If denominator is zero, proportions are null and status is insufficient evidence.

- Without a reference, `reference_proportion` and `difference_pp` are null. Multiple observed groups yield descriptive-only unless another explicit review finding exists. A single observed group yields review-needed, not proof of bias.

- With a reference, report differences in percentage points, without a universal 'FDA 5%' rule. Missing expected groups warrant review. Other threshold policies and uncertainty calculations require a separate documented decision; don't invent them in v1.

- `priority` is an issue-triage label, never a clinical-risk estimate.

- UI-friendly explanations may supplement these values but cannot overwrite them.

W02 adds separate `validate_input` and `screen_identifiers` result types to `contracts.py`; Shivam integrates them only after their fields/coverage semantics are documented. W03 similarly documents preview input/output before UI integration.

Branch and merge workflow

Each person uses a separate clone (or isolated worktree), not the same checkout. Repo write access is required for pushing; Swathi's GitHub username is needed only if an invitation is required. Creating this plan does not invite or message her.

Start each task from latest remote main:

```
git fetch origin
git switch -c shivam/audit-workspace origin/main
# Swathi uses: git switch -c swathi/representation-engine origin/main
```

Commit only your task's files. Push your own branch and open one PR per task. Include the task ID, behavior change, test commands/results, and limitations. Do not push directly to main or reuse the already merged `fix/audit-correctness` branch.

When another PR merges, update a clean task branch with `git fetch origin` then `git merge origin/main`. Do not rebase a branch another person is using or force-push over their work. If edits overlap, let the file owner resolve the conflict and rerun the affected tests.

Recommended merge order:

1. W01 and S01 may merge in either order if S01 remains fixture-driven and does not claim backend integration. 2. W02 after W01; W04 can proceed independently after W01. 3. S02 after W01/W02/S01. This is the first integrated release gate. 4. W03 -> S03 for validated previews; S04 follows the stable integration. 5. S05 and W05 finalize the demo and independent validation.

After a merge, use a new branch from updated main for the next task. Integration owner: Shivam. No other person needs to edit `app_complete.py` to ship their independent task.

Release gates and scope control

First release must pass: original-data regressions, unavailable-harm regressions, single-group/no-demographic/all-null cases, deterministic repeated results, malformed input handling, honest privacy coverage, and usable offline demo. Exported report must match displayed measurements.

Existing tests: `.venv/bin/python -m pytest -q tests`. New service tests must run without Ollama, downloads, or network; UI tests use deliberate model-unavailable mocks. A green test run is not evidence of clinical safety or privacy certification.

If time is short: finish W01/S01/W02/S02 plus W05, then polish S05. Keep unsafe transformations disabled. Defer full RAG recovery, synthetic generation, recruitment plans, deployment roadmaps, real model-performance evaluation, and a framework rewrite. The saved backlog retains those open issues.

Copyable handoffs

[SWATHI - W01] Start `swathi/representation-engine` from `origin/main`. Implement `contracts.py` and `representation.py` under `src/services`, plus `tests/analysis`. Follow proposed v1 above; do not edit the UI or shared dependency files. Return descriptive evidence, not FAIR/BIASED verdicts. Test empty/no demographic, all-null, one group, nonbinary, numeric-coded, missing reference group, and invalid reference cases. Preserve the input. Deliver a PR with test evidence and any proposed contract changes.

[SHIVAM - S01] Start `shivam/audit-workspace` from `origin/main`. Build a clear audit workspace in `src/ui` against fixture results matching v1, with fixtures confined to explicitly identified demo/test paths. Provide immediate demo access, column selection, prioritized evidence, cohort counts/missingness, unknown states, and accessible navigation. Disable legacy unsafe mutation controls. Do not modify Swathi's services/utilities. Preserve and extend UI regression tests. Integrate real service output in S02 after backend PRs merge.